

Exercício de Fixação — TADs e Orientação a Objetos

Página / Slide 23

Aluno: Rodrigo Del Padre

RA: 24042092-2

Curso: Engenharia de Software (6º Semestre)

Turma: B

Disciplina: Paradigmas de Programação

Referência: R. Sebesta, 11ª ed., Caps. 11 e 12

Trecho 5 — Java

Código analisado:

```
class A {  
    static String quem() { return "A"; }  
}  
class B extends A {  
    static String quem() { return "B"; }  
}  
A x = new B();  
System.out.println(x.quem());
```

(a) Qual é a saída?

A

(b) A decisão foi tomada na compilação ou na execução?

Decisão tomada na COMPILAÇÃO (Compile-time)

Conceito Teórico: Métodos Estáticos de Classe, Ocultação de Métodos (*Method Hiding*) e Vinculação Estática via `invokestatic` (Sebesta, 11ª ed., §12.3.5, p. 497; Slides 15 e 17).

- **Métodos Estáticos Não São Polimórficos:** Métodos qualificados com `static` pertencem à classe e não às instâncias individuais. Eles não participam da tabela virtual (*vtable*) e não sofrem despacho dinâmico.
- **Ocultação (*Hiding*):** Em Java, a subclasse não sobrescreve um método estático; ela apenas o oculta.
- **Resolução Estática:** Mesmo com a sintaxe `x.quem()`, o compilador avalia exclusivamente o tipo estático da variável referenciadora (`x` é do tipo `A`) e emite a instrução de bytecode `invokestatic A.quem()`. A JVM executa `A.quem()` diretamente sem consultar o tipo do objeto no *heap*.

Trecho 6 — Go

Código analisado:

```
type Animal struct{  
    func (Animal) Som() string { return "..."}  
    func (a Animal) Falar() string {  
        return "faz " + a.Som()  
    }  
}  
type Cao struct{ Animal }
```

```
func (Cao) Som() string { return "au" }  
  
fmt.Println(Cao{}.Falar(), Cao{}.Som())
```

(a) Qual é a saída?

```
faz ... au
```

(b) A decisão foi tomada na compilação ou na execução?

Decisão tomada na **COMPILAÇÃO** (Compile-time)

Conceito Teórico: POO sem Herança de Classes, Embutimento de Estruturas (*Struct Embedding*), Promoção de Métodos e Vinculação Estática de Receptores (Sebesta, 11ª ed., §12.3.1–12.3.3; Slides 18 e 19).

- **Composição por Embutimento:** Go não tem herança nem palavras-chave como `extends`. Ao definir `type Cao struct{ Animal }`, a estrutura `Animal` é embutida anonimamente, e seus métodos são promovidos para `Cao`.
- **Ausência de Métodos Virtuais em Structs:** O método `Falar` possui como receptor estático (a `Animal`). Quando ele executa `a.Som()`, o compilador Go vincula estaticamente a chamada à função `Animal.Som` com base no tipo do receptor `a` (que é `Animal`). Não há despacho dinâmico implícito nem ponteiro virtual para `Cao`. Logo, `Cao{}.Falar()` retorna `"faz ..."`.
- **Chamada Direta:** A chamada `Cao{}.Som()` resolve diretamente para o método com receptor `Cao`, retornando `"au"`.
- **Conclusão:** Ambas as chamadas são resolvidas em tempo de compilação, gerando `"faz ... au"`.